LYNQ

Plataforma inteligente de empleabilidad: funcionalidades y decisiones técnicas

Jorge Flores · jfflores@udesa.edu.ar
Maestría en Inteligencia Artificial, Universidad de San Andrés
Proyecto del *New Business Launch* (NBL), en conjunto con el Executive MBA
Tatiana Ibañez · tibanez@udesa.edu.ar Melani Huetagoyena Rija · mhuetagoyena@udesa.edu.ar

Octubre de 2026 · versión 3.1.5

1. Qué es LYNQ

LYNQ surgió como proyecto final del programa *New Business Launch* de la Universidad de San Andrés. Es una propuesta de colaboracion entre dos programas: el Executive MBA. El cual definió el caso de negocio (cliente, propuesta de valor, MVP y modelo de negocios) Y la Maestría en Inteligencia Artificial que aporta el conocimiento tecnico para desarrollar la plataforma. Su propósito se resume en el lema *"que buscar trabajo no sea un trabajo"*. LYNQ se presenta como la primera plataforma inteligente de empleabilidad en LATAM, un marketplace bidireccional que transforma la búsqueda laboral en un proceso predictivo, transparente y automatizado.

El problema que aborda tiene dos aristas. Para el candidato, la búsqueda de empleo está fragmentada (el 100 % de los entrevistados usa más de tres portales), exige cargar el perfil repetidas veces y adaptar el CV a mano, y las búsquedas activas se extienden entre 3 y 6 meses; además, ninguno de los entrevistados conocía el rango salarial antes de recibir una oferta. Para la empresa, los datos salariales disponibles están desactualizados o son costosos, no existe un *time-to-fill* predictivo, y el screening manual consume entre el 30 % y el 50 % del tiempo de RRHH. En 18 entrevistas (12 candidatos y 6 empresas), el 83 % de los candidatos y 5 de 6 responsables de RRHH confirmaron la relevancia de estos problemas. Los aspectos más valorados fueron la transparencia salarial y el LYNQ Score del lado B2C, y *shortlist* y el *time-to-fill* del lado B2B.

Cliente objetivo. En la fase 1 el foco se puso en profesionales de 22 a 40 años con perfiles no técnicos (administración, legales, comercial, marketing, finanzas) y en startups y scale-ups de 150 a 400 empleados, que contratan de 3 a 8 posiciones al mes y buscan reducir el *time-to-fill* de 6 semanas a menos de 3.

Modelo de negocios. Combina freemium B2C con SaaS B2B: LYNQ Free (USD 0), LYNQ Pro (USD 9,99/mes), Business Starter/Growth/Enterprise (USD 290, 790 y desde 1.900/mes) y LYNQ Data API (desde USD 500/mes). La transparencia salarial se mantiene gratuita. Se estima que el punto de equilibrio se alcanza con 50 cuentas B2B y 1.300 suscriptores Pro.

2. Funcionalidades

- **Para el Candidato.** Incluye registro, carga de CV (PDF/DOCX) y extracción automática de skills y *similarity tags*. Dispone de un buscador unificado de avisos propios y scrapeados (Bumeran y Computrabajo). **LYNQ Score** de 0 a 100 por aviso, y postulación con el CV seleccionado. Ofrece **CV Tailor**: un agente conversacional que adapta el CV a un aviso, sugerencias de *upskilling* con cursos de Udemy, y analíticas personales: posición entre los postulantes de un aviso, salario esperado frente al mercado y a los pares, *benchmark* entre perfiles similares, cobertura de skills y qué skills faltantes abren más avisos.
- **Para la Empresa.** Permite publicar y gestionar avisos, ver el listado de candidatos ordenado por score con una explicación generada (fortalezas y preocupaciones), consultar un *time-to-fill* estimado a partir de avisos similares, y mostrar en un panel el estado, las postulaciones y el score de cada aviso.

Finalmente, la plataforma expone una vista de mercado con los avisos abiertos, el top 10 de skills más demandadas con su variación semanal, los salarios por categoría y modalidad, y los avisos publicados por semana (historial de 12 semanas).

3. Arquitectura

LYNQ se construye como un monorepo de ocho servicios en GitHub, desplegados en Kubernetes (EKS), más dos frontends en Cloudflare (Figura 1).

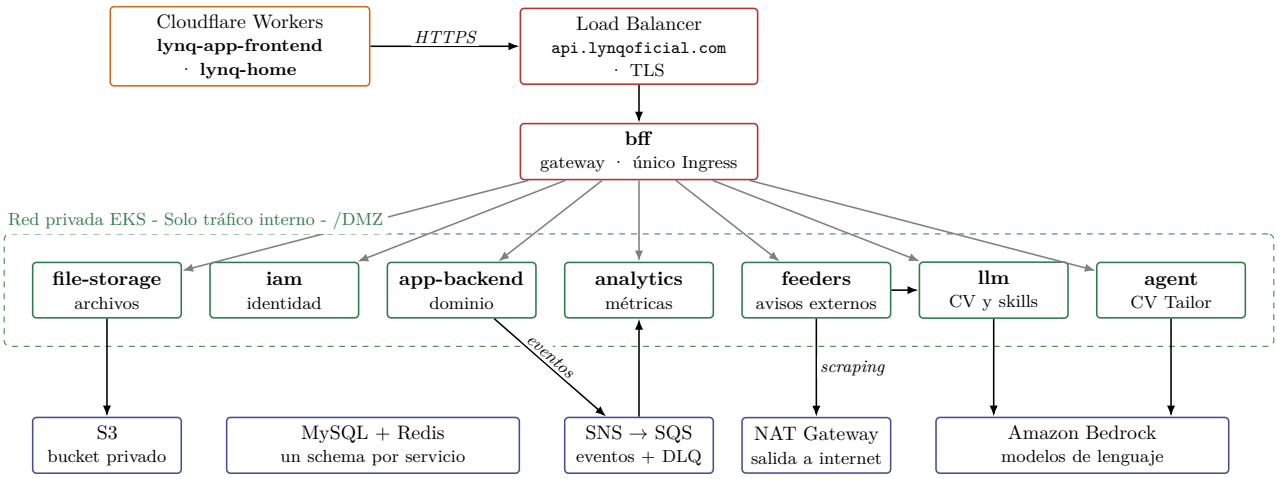


Figura 1: Solo las aplicaciones instaladas en Cloudflare y el Load Balancer reciben tráfico de internet; el BFF es el único servicio expuesto y redirige a los siete microservicios, accesibles únicamente por la red interna de Kubernetes. Todos los servicios de dominio usan MySQL y Redis; solo `file-storage` interactúa con S3, solo `llm` y `agent` invocan a Bedrock, y `feeders` es el único que sale a internet cuando obtiene las nuevas postulaciones de trabajo de los portales externos

Servicio	Stack	Puerto	Responsabilidad
lynq-bff	Spring Boot	8080	Único Ingress. Verifica la firma del JWT, redirige cada ruta a su servicio y orquesta los flujos compuestos (importación de CV, CV Tailor).
lynq-iam	Spring Boot	8080	Usuarios, roles, emisión y validación de JWT, refresh tokens en Redis.
lynq-app-backend	Spring Boot	8080	Avisos, empresas, CVs, postulaciones, LYNQ Score, publicación de eventos de dominio e ingesta de avisos externos.
lynq-file-storage	Spring Boot	8080	Metadatos de archivos y URLs pre-firmadas de S3. Es el único servicio con credenciales de S3.
lynq-analytics	Spring Boot	8080	Consume eventos, proyecta un read model y sirve las analíticas; snapshot diario.
lynq-llm	FastAPI	8084	Parseo de CV, extracción de skills y tags, explicaciones, traducción y cursos. Bedrock Nova Lite.
lynq-agent	FastAPI + LangGraph	8090	CV Tailor: grafo multiagente sobre Bedrock Nova Pro, con persistencia y traza propias.
lynq-feeders	FastAPI	8089	Scraping de Bumeran y Computrabajo, enriquecimiento con lynq-llm y verificación de vigencia.

Cuadro 1: Servicios del monorepo. La infraestructura se gestiona como código con Terraform (VPC, EKS, S3, SNS/SQS, IAM) y Helm (deployments, CronJobs); LocalStack y docker-compose replican el stack para desarrollo local.

4. Autenticación y autorización

Se optó por autenticacion propia. Esta plataforma es una propuesta y no un producto final, e incluir el login con Google habría sumado integraciones que no aportaban al objetivo del trabajo; el botón de "Login with Google"quedó visible en la aplicación como señal de que extenderlo es posible, pero el flujo OAuth 2.0/OIDC no está implementado: no hay *authorization code*, *id_token* ni alta automática de usuarios. El sistema actual se basa en un esquema de dos tokens emitido por `lynq-iam`:

1. **Registro** (`POST /auth/register`): requiere `{username, password, email, role}`, con rol `R_CANDIDATE` o `R_COMPANY`. La contraseña se almacena con BCrypt (*strength* 10).
2. **Login** (`/auth/login/email` o `/username`): devuelve un *access token* JWT y un *refresh token*. El JWT contiene `jti`, `sub` (userId), `username`, `email`, `roles`, `iat` y expira a los 15 minutos. El refresh token tiene un TTL de 30 días.
3. **Refresh** (`POST /auth/refresh` con `Authorization: Bearer {refreshToken}`): si el token existe en Redis, se relee el usuario de MySQL (de modo que los roles se actualizan) y se emite un JWT nuevo. El refresh token no rota.
4. **Relay en el BFF.** Se aplican tres filtros en orden: `RequestUuidFilter` (exige `lynq-request-uuid`, el id de correlación que viaja a todos los servicios para un mejor tracing de las acciones de los usuarios), `AuthHeaderExistenceFilter` (retorna 401 si falta `Authorization`) y `JwtSignatureFilter`, que verifica la firma localmente con el mismo secreto y construye un `LynqUserPrincipal`. Las únicas rutas anónimas son registro, login y los *checks* de disponibilidad; `/auth/refresh` requiere header pero no firma.
5. **Validación.** Los servicios de dominio no verifican la firma directamente. En su lugar invocan `GET /auth/user-info` de IAM con el bearer reenviado por el BFF y construyen el principal con los roles devueltos. Esta llamada se realiza en cada request, a cambio de que el secreto resida únicamente en IAM
6. **Autorización.** `@HasRole(CANDIDATE|COMPANY)` se declara en el servicio propietario del dato, bajo `/dmz/**`. Las rutas `/internal/**` (ingesta, replay, snapshot, score en lote) no pasan por el BFF y están protegidas por un `InternalTokenFilter` con el header `lynq-internal-token`.

En el frontend, con *remember me* el refresh token se guarda en `localStorage` y la sesión se rehidrata al recargar; sin esa opción, todo reside en `sessionStorage`. Ante un 401, `authFetch` refresca el token y reintenta una vez.

5. LYNQ Score y similitud

5.1. LYNQ Score

El score se pensó inicialmente solo sobre las skills, es decir, cuántas de las que pide el aviso aparecen en el CV. En la práctica eso resultó demasiado literal, porque existen muchas herramientas que resuelven el mismo problema y un candidato que domina una de ellas rara vez tiene dificultades con otra equivalente: quien sabe usar Kafka no debería tener problemas para implementar la misma solución sobre SNS/SQS. Para capturar eso se introdujeron las *similarity tags*, que no describen herramientas sino el tipo de problema que la persona sabe resolver, y el score pasó a considerar las dos coberturas.

El LYNQ Score se calcula en `lynq-app-backend` en cada request debido a que los CV o las ofertas laborales pueden ser editadas y, en consecuencia, los tags o skills cambian. El cálculo queda centralizado en un solo lugar. Se define V como el vocabulario del candidato, es decir, la unión de sus skills y sus *similarity tags* normalizadas (*trim* y minúsculas); S como las skills que solicita el aviso y T como sus tags. La cobertura de un conjunto requerido R se define como

$$\mathrm{cov}(R) = \mathrm{round}\left(100 \cdot \frac{|R \cap V|}{|R|}\right), \qquad \mathrm{cov}(\varnothing) = 0,$$

y el score como

$$\text{score} = \begin{cases} 0 & \text{si } V = \varnothing \text{ o } S \cup T = \varnothing, \\ \max\left(\text{cov}(S),\ \text{cov}(T)\right) & \text{en otro caso.} \end{cases}$$

No se aplican pesos: prevalece la mejor de las dos coberturas. Las skills son términos específicos ("Spring Boot"), mientras que las tags son conceptos en inglés que lynq-llm genera para avisos y CVs con el mismo prompt ("backend development"), lo que permite que $\text{cov}(T)$ identifique candidatos cuyo CV usa palabras diferentes. El score se incluye en el evento `ApplicationSubmitted` y se expone también en `POST /internal/score/batch` para el snapshot de analytics.

5.2. Similitud entre avisos y entre candidato y aviso

Las comparaciones contra avisos o candidatos similares y la similitud se calcula en `lynq-analytics` a partir de los tags, ponderando cada una con una variante de IDF que se recalcula en cada snapshot sobre los N avisos del read model:

$$w(t) = \ln\left(\frac{N+1}{\text{df}_t + 1}\right),$$

donde df_t (*document frequency*) es la cantidad de avisos que contienen la tag t. Se trata de la fórmula de *inverse document frequency* (IDF) clásica: un tag es más valioso cuanto más raro es en el corpus. Por ejemplo, con $N = 200$ avisos, un tag presente en 180 de ellos pesa poco, mientras que uno presente en 5 pesa unas 35 veces más:

Tag	Avisos que la contienen (df_t)	Peso $w(t)$
`teamwork`	180	$\ln(201/181) \approx 0{,}10$
`backend development`	40	$\ln(201/41) \approx 1{,}59$
`kubernetes`	5	$\ln(201/6) \approx 3{,}51$

Así, dos avisos que solo comparten `teamwork` no se consideran parecidos, ya que casi todos lo tienen; en cambio, dos que comparten `kubernetes` sí. Sin esta ponderación, ambas coincidencias tendrían el mismo peso y el conjunto de avisos similares contendría mucho ruido. El +1 en numerador y denominador es un suavizado que evita el logaritmo de cero. Los pesos se recalculan en cada snapshot diario y se guardan en `tag_frequency`; un tag desconocido recibe el peso máximo observado. Se llama m a la mediana de los pesos por ocurrencia (cada tag cuenta df_t veces), que representa el valor de un tag típico.

Aviso contra aviso (time-to-fill, salario de la posición). Se utiliza el índice de Jaccard ponderado entre los tags A del aviso de referencia y los tags B del otro aviso,

$$J(A,B) = \frac{\sum_{t \in A \cap B} w(t)}{\sum_{t \in A \cup B} w(t)}, \qquad \theta_k = \min\left(1,\ \frac{k \cdot m}{\sum_{t \in A} w(t)}\right),$$

y se admite el aviso si $J(A,B) \geq \theta_k$, donde k es la cantidad de tags típicos que se exige compartir y θ_k el umbral que resulta para ese aviso de referencia.

Como ejemplo, se toman los pesos de la tabla anterior, más `java` (1,20), `sql` (0,90), `microservices` (2,30), `excel` (0,60) y `communication` (0,15). El aviso de referencia es $A = \{$`backend development`, `kubernetes`, `java`, `teamwork`$\}$, con $\sum_{t \in A} w(t) = 6{,}40$, y se lo compara con otros dos avisos que comparten exactamente dos tags con él:

Aviso	Tags	En común	Jaccard simple	J ponderado
B	`backend development`, `kubernetes`, `sql`, `microservices`	2	$2/6 = 0{,}33$	$5{,}10/9{,}60 = 0{,}53$
C	`teamwork`, `java`, `sql`, `excel`, `communication`	2	$2/7 = 0{,}29$	$1{,}30/8{,}05 = 0{,}16$

Contando tags, B y C se parecen a A casi en la misma medida. Al ponderar, B comparte las dos tags de mayor peso y C las dos de menor peso: con $m = 1{,}2$ el umbral es $\theta_2 = \min(1,\ 2 \cdot 1{,}2/6{,}40) = 0{,}375$, por lo que B entra al conjunto de avisos similares y C queda fuera.

El umbral θ_k no es un número fijo. Se calcula para cada aviso de referencia y significa siempre lo mismo, compartir el equivalente a k tags de peso típico, con $k = 2$ (`thresholdTags`). Como J es una fracción sobre la unión, ese mismo requisito se traduce en un valor distinto según cuántos tags tenga A:

Aviso de referencia	$\sum_{t \in A} w(t)$	θ_2 con $m = 1{,}2$
4 tags (el A del ejemplo)	6,40	0,375
10 tags	15,0	0,16
2 tags	2,0	1,0: debe coincidir en todo

Un umbral fijo de 0,5 haría que un aviso con doce tags casi nunca encontrara similares y que uno con dos tags encontrara cualquier cosa. Si con $k = 2$ quedan menos de 5 avisos admitidos (`minSample`), se reintenta con $k = 1$ (`fallbackThresholdTags`) y la respuesta se marca como `fallback`; si aun así no se alcanzan 5, la métrica devuelve `insufficientData` y el frontend indica la falta de datos en lugar de mostrar un número. Los avisos similares no se muestran en la aplicación; son la muestra sobre la que se calculan el *time-to-fill* y el salario de la posición.

Candidato contra aviso (salario esperado de pares, *benchmark*). Se utiliza el *overlap* ponderado sin normalizar, porque el CV tiende a tener muchas más tags que un aviso y el índice de Jaccard lo penalizaría:

$$O(A, C) = \sum_{t \in A \cap C} w(t), \qquad \theta_k = k \cdot m.$$

Se aplican la misma k y el mismo *fallback*; dado que el *overlap* no está normalizado, el umbral es directamente $k \cdot m$. En ambos casos, los empates en el score se resuelven por la cantidad de skills compartidas.

6. Manejo de archivos

Subir los archivos directamente a S3 fue la idea desde el inicio, pero en los primeros commits era lynq-app-backend quien administraba las rutas, con una estructura de carpetas predefinida. Mantener cada *filepath* desde el dominio resultó complicado y agrandaba el código del servicio con lógica que no le correspondía, así que esa responsabilidad se separó en lynq-file-storage, tomando como referencia un microservicio similar ya existente en un entorno profesional.

El contenido de los archivos nunca pasan por un servicio de LYNQ. `lynq-file-storage` almacena solo metadatos en MySQL (`stored_files`: `id`, `file_name`, `content_type`, `s3_key`, `status`, `owner_user_id`) y delega el contenido de los archivos a un bucket S3 privado (`lynq-oficial-file-storage`) mediante URLs pre-firmadas con expiración de 15 minutos. Es el único servicio con credenciales de S3; `lynq-app-backend` guarda únicamente el id del archivo (`users`, `companies`, `user_resumes`, `user_application_job`).

El flujo para importar un CV, orquestado por el BFF, consta de cinco pasos:

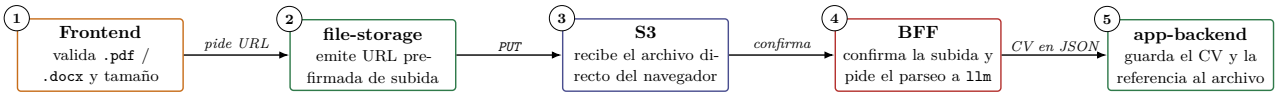


Figura 2: Importación de un CV. El contenido de los archivos van del navegador a S3 sin pasar por ningún servicio; el resto del flujo mueve solo ids, URLs pre-firmadas y el CV ya parseado.

- El frontend valida la extensión (`.pdf`, `.docx`) y el tamaño (10 MB) antes de iniciar el proceso.
- File-storage registra el archivo como `PENDING` y devuelve una URL pre-firmada para la subida.
- El navegador sube el archivo directamente a S3 con esa URL.
- El BFF confirma la subida (file-storage verifica en S3 y cambia el estado a `AVAILABLE`) y envía el archivo a `lynq-llm`, que detecta el idioma y extrae skills y tags.

- El backend guarda el CV, el idioma, las skills y la referencia al archivo. Si algo falla en el proceso, el archivo se elimina.

Ownership. El propietario se determina a partir del token resuelto contra IAM. Solo él puede confirmar, consultar o eliminar un archivo (403 en caso contrario). Las descargas, en cambio, no se restringen al propietario de forma intencional: el id del archivo funciona como una *capability*, y es el backend quien decide qué ids mostrar (la empresa ve el CV con el que el candidato se postuló a su aviso). Los CVs adaptados por CV Tailor se suben por el mismo mecanismo de URL `PUT` pre-firmada.

7. Eventos de dominio

En una primera versión del diseño la publicación de eventos usaba el patrón *outbox*[1], con una tabla en el backend y un proceso que la drenaba periódicamente. Se descartó para simplificar: el tiempo que iba a demandar mantener esa pieza era tiempo que convenía dedicar a lynq-agent, y el mecanismo de replay que se describe más abajo cubría el mismo riesgo con mucho menos código.

Analytics no lee directamente la base de datos del backend. Escucha los eventos, los guarda y los proyecta en un read model con la estructura que necesitan las consultas. Esto reduce el acoplamiento con el esquema transaccional y permite modificar o añadir un gráfico reproyectando desde los eventos guardados.

7.1. Publicación

Existen siete tipos de eventos sobre tres agregados: `JobPostPublished`, `JobPostUpdated`, `JobPostClosed`, `JobPostReopened`, `ApplicationSubmitted` (con el score ya calculado), `CandidateSkillsUpdated` y `CandidateExpectedSalaryUpdated`. Las decisiones de publicación son las siguientes:

- **Delivery *at-most-once*.** El servicio envía el evento a `DomainEventPublisher` dentro de la transacción, y `DomainEventRelay` lo envía a SNS. Si el envío falla, el evento se pierde; el replay se encarga de recuperarlo.
- **Ids determinísticos.** `eventId` es un UUIDv5 del tipo, del `aggregateId` y del hecho que el evento describe (un cierre se identifica por sus fechas de apertura y cierre, una postulación por su id, las skills por el conjunto completo). Publicar el mismo hecho dos veces produce el mismo id.
- **Replay.** Si algo se pierde se recupera con `POST /internal/events/replay`, que recorre `job_posts`, `users` y `user_application_job` en páginas de 200 y reemite el estado actual en lotes de 10. Dado que los ids son determinísticos, lo ya existente no se guarda debido al `UNIQUE` del consumidor, y lo nuevo sí; ejecutarlo dos veces no tiene efectos secundarios.

7.2. Consumo y proyección

SNS (`lynq-domain-events`) distribuye a la cola SQS `lynq-analytics-events`. El listener ejecuta el `INSERT` en `domain_events` (`event_id CHAR(36) UNIQUE`, `event_type`, `aggregate_type`, `aggregate_id`, `payload JSON`, `occurred_on`, `received_on`) y la proyección al read model (`job_posts`, `job_post_skills`, `job_post_tags`, `candidates`, `candidate_skills`, `candidate_tags`, `applications`). La idempotencia se garantiza por el `UNIQUE` de `event_id`: un duplicado falla el insert y se ignora. Los eventos fuera de orden se resuelven con marcas de tiempo por entidad (`details_occurred_on`, `status_occurred_on`, `skills_occurred_on`, `salary_occurred_on`): uno más antiguo que lo ya proyectado se descarta, pero permanece en `domain_events`; una actualización que llega antes del publish se rechaza y SQS la reintenta. Si la proyección falla, el evento se revierte y vuelve a la cola.

[1] Patrón en el que el evento se guarda en una tabla de la misma base de datos, dentro de la transacción del dominio, y un proceso aparte lo publica después; garantiza que ningún evento se pierda a costa de una tabla y un job adicionales.

7.3. Tres niveles de actualización

- **Nivel 1, al llegar el evento**: el read model se actualiza en segundos. El costo es la consistencia eventual: la posición de un candidato puede no reflejar una postulación recién realizada.
- **Nivel 2, al consultar**: las métricas por aviso se calculan sobre el read model en cada request (decenas o cientos de filas) y se cachean en Redis durante una hora.
- **Nivel 3, snapshot diario**: un CronJob (08:00 UTC, después del feeder de las 06:00) invoca `POST /internal/snapshot`, que recalcula las frecuencias de tags, escribe una fila por día y dimensión en `job_daily_stats`, `skill_daily_demand` y `category_daily_salary` y construye `candidate_daily_benchmark` solicitando scores al backend en lote. Este nivel existe solo para los gráficos con fecha en el eje; si un día el job no se ejecuta, el gráfico muestra datos faltantes en lugar de interpolar.

7.4. Métricas expuestas

Endpoint (`/dmz/analytics`)	Rol	Qué calcula
`GET /job/{id}/time-to-fill`	COMPANY, dueño	Mediana, p25 y p75 (interpolación lineal) de los días entre apertura (o reapertura) y cierre de avisos similares. Excluye y cuenta aparte los `EXPIRED_BY_POLICY`. Con $N < 5$ devuelve la distribución global.
`GET /job/{id}/standing`	CANDIDATE, postulado	Sobre `applications.lynq_score`: rank $= 1 + \#\{\text{scores} > s\}$ y percentil $= 100 \cdot (\text{below} + \text{tied}/2)/\text{total}$; histograma por bandas; mediana del aviso si $N \geq 5$.
`GET /job/{id}/salary`	cualquiera	`positionSalary`: mediana, p25 y p75 del punto medio del rango de avisos similares, en la moneda del aviso. `peersExpectedSalary`: lo mismo sobre el salario esperado de candidatos similares. Se usa la mediana y no la media porque los salarios scrapeados mezclan períodos y contienen errores.
`GET /candidate/me/benchmark`	CANDIDATE	Del snapshot: *market fit* (mediana del score contra los avisos relevantes), *reach* (% de avisos con score ≥ 60), percentil y cuartiles entre pares, cobertura de skills, serie de 90 días y *skill unlocks*: cuántos avisos pasarían el umbral si el candidato tuviera cada skill faltante.
`GET /market?currency=`	cualquiera	Avisos abiertos y con salario, top 10 de skills con variación semanal, salario por categoría y modalidad, publicados por semana.
`GET /company/me/jobs`	COMPANY	Avisos propios con estado, postulaciones y score mediano. Es el único cache que se invalida por evento.

Cuadro 2: Analíticas servidas por lynq-analytics.

8. CV Tailor: el agente de adaptación de CV

La primera versión de CV Tailor fue un agente ReAct[2] con dos herramientas, `find_evidence` para buscar información en el CV y `edit` para modificarlo, y durante las pruebas aparecieron dos problemas: El primero fue que reordenaba la experiencia laboral para mostrar primero la más relevante para la posición, algo que suena correctamente pero que no se corresponde con la forma en que se escribe un CV, donde el orden es cronológico. El segundo, fue que ante una entrada poco relevante optaba por borrarla, cuando la intención siempre había sido parafrasearla. De este modo quedó claro que el problema no se resolvía pidiéndole al modelo que no invente ni que no borre, sino separar el problema en tratar de identificar la intencion del usuario, proponer, juzgar y luego, aplicar la modificacion a traves de codigo deterministico.

[2]Patrón en el que el modelo alterna razonamiento y acciones: decide qué herramienta invocar, observa el resultado y vuelve a razonar hasta dar por terminada la tarea.

8.1. El grafo

Cada turno de conversación se ejecuta como un grafo de LangGraph (Figura 3) sobre Amazon Bedrock (Nova Pro en producción; qwen2.5:7b en Ollama para desarrollo). El estado del turno (`TurnContext`) contiene el aviso, el CV base, el CV actual, los últimos cuatro pares del hilo, las recomendaciones del turno anterior y los *statements*: todo lo que el candidato dijo en la conversación más lo que ya confirmó.

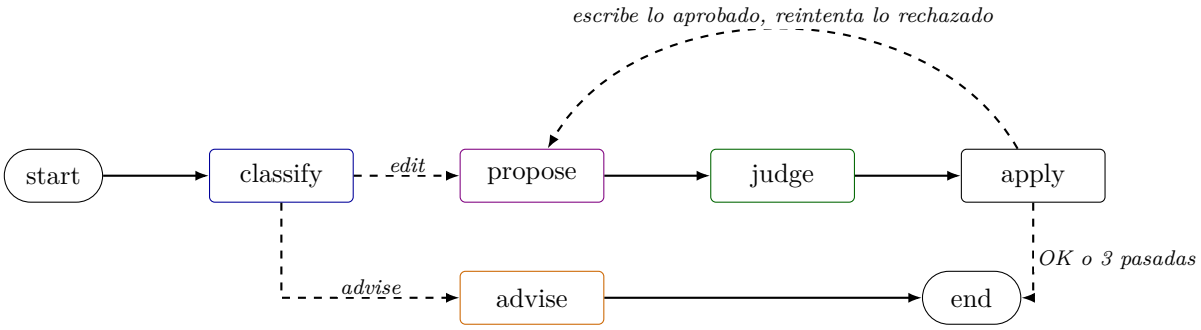


Figura 3: Grafo de un turno de CV Tailor. **classify**: agente de intención; **advise**: agente de recomendación; **propose**: agente de edición; **judge**: agente juez; **apply**: código, sin modelo. La rama `advise` no tiene ningún nodo que escriba, de modo que una consulta no puede modificar el CV por construcción.

1. **classify** (agente de intención). Realiza una llamada y responde una sola palabra, `edit` o `advise`, leyendo el mensaje junto con los últimos cuatro del hilo. Un "sí" a una skill por la que CV Tailor preguntó se clasifica como `edit`. Si el modelo falla o responde otra cosa, se asume `edit`, de modo que nunca se pierde un turno. Es la única decisión de ruteo que toma un modelo, y puede ejecutarse en un modelo distinto (`BEDROCK_INTENT_MODEL_ID`).
2. **advise** (agente de recomendación). Devuelve `reply`, `warnings` y `recommendations` como datos `{id, section, entry, what}`; su esquema no contiene campos para modificar el CV. Las recomendaciones se numeran y persisten en el mensaje, para que un "hacé la 2" en el turno siguiente tenga un referente.
3. **propose** (agente de edición). Recibe el aviso, el CV sin `personal_info`, los *statements* y la lista de *gaps*, y responde un `EditProposal` con *structured output*: `summary`, entradas de experiencia identificadas por empresa y puesto (nunca por índice) con `description` y `achievements`, los tres *buckets* de skills (`technical`, `tools`, `soft`), `confirmed` (lo que el candidato afirmó sobre sí mismo en este turno) y `reply`. No usa tools ni loops: responde un formulario.
4. **Guard de forma** (código, antes del juez). `plan` resuelve cada parte en su lugar del CV base y rechaza sin consultar al modelo: `cut` si se pierden líneas, números o tecnologías; `moved` si una línea cambió de posición (Jaccard de tokens $\geq 0{,}75$ para considerarla la misma); `unbacked` si aparece un nombre propio o una skill que no está en el CV ni en lo que dijo el candidato; `unknown_entry` si la entrada no existe. Un *bucket* de skills puede crecer al final, pero nunca achicarse ni reordenarse.
5. **judge** (agente juez). Lee cada parte junto a su original, con las líneas numeradas y contadas, más `candidate_statements` y `this_turn` (qué preguntó CV Tailor y qué contestó el candidato). Debe citar la evidencia antes de aprobar, copiada del CV o de lo dicho por el candidato, nunca de la propuesta ni del aviso, ya que solicitar una skill no implica tenerla. Devuelve un `Verdict` por parte con `ok`, `kind` y `reason`; los `kind` de rechazo son `language`, `dropped_content`, `reordered`, `dropped_skill`, `invented`, `unsupported_skill`, `wording` y `padding`. Una parte que el juez omite queda `unjudged` y no se aplica.
6. **apply** (código). `commit` escribe en el CV solo lo aprobado y registra cada cambio. Si quedaron rechazos y aún quedan pasadas (`MAX_PASSES` = 3), el grafo vuelve a `propose` con el motivo de cada rechazo en el hilo; es la única vuelta atrás del grafo. Que la vuelta salga de `apply` y no del juez es intencional: lo aprobado se escribe en cada pasada, y la siguiente propuesta se hace sobre el CV ya actualizado y únicamente sobre lo rechazado. Al terminar, el candidato recibe el `reply` del

modelo y, por separado, un aviso generado por código con lo que no se aplicó y por qué, de modo que el chat no describe cambios que el CV no contiene.

Cómo sabe `apply` qué modificar. La propuesta no es texto libre sino un formulario, y cada campo del `EditProposal` corresponde a una ubicación del CV: `summary` solo puede ir al resumen, una entrada de experiencia se identifica por empresa y puesto, y `skills.technical` es ese *bucket* y ningún otro. El modelo no puede pedir "cambiá la tercera línea" ni agregar una sección, porque el esquema no tiene campos para eso. `plan` convierte el formulario en *partes*, cada una con su ruta en el JSON del CV, el texto original y el propuesto; el juez devuelve un veredicto por parte con el mismo id, y `commit` recorre la lista: escribe en su ruta las aprobadas y arma, con una plantilla, el aviso de las rechazadas. Un turno con dos partes se vería así:

id	Ruta en el CV	Original	Propuesto	Veredicto
1	`work_experience[Acme].description`	Built services on Kubernetes.	Built services on Kubernetes. – Built the payment workers in Go	`ok`
2	`skills.technical`	Java, Postgres, Kubernetes	Java, Postgres, Kubernetes, Go	`ok`

Si el modelo escribiera el CV final directamente, una parte rechazada no se podría separar de una aprobada, y nada garantizaría que no modificó `personal_info`, la educación o las fechas. Con partes direccionadas, lo rechazado queda fuera, y lo que el esquema no contempla no puede modificarse.

Gaps y evidencia del candidato: Al crear la conversación, la aplicación compara las skills del aviso (`extractedSkills`, obtenidas vía `lynq-llm /skill-enhance`) con el CV y los *statements*, y construye el saludo sin llamar al modelo: lista hasta 4 gaps y pregunta sí el usuario tiene alguna experiencia en esos gaps, en qué puesto y qué hizo el candidato con cada uno. Una skill que el candidato confirma entra al *bucket* en el mismo turno; a una entrada de experiencia entra recién cuando dice dónde la usó y para qué. Lo confirmado se persiste en `conversation.confirmed` y vuelve como evidencia en todos los turnos siguientes, dentro o fuera de la ventana de contexto.

Persistencia de trazas: `lynq_agent` guarda la conversación: snapshots del aviso y del CV, precios por token, `run_token`, totales de costo, `message`, `resume_version` (una por turno con su lista de cambios) y `trace_span` (un span por llamada al modelo con tokens, costo y latencia, y uno por pasada de `apply` con el veredicto). El BFF expone la funcionalidad en `/resume/{resumeId}/tailor/{jobId}`, `/tailor/{id}/turn` y `/tailor/{id}/apply`, que renderiza el CV adaptado como archivo y lo deja listo para postular.

9. Resumen de decisiones y deuda

Decisión	Por qué	Costo
BFF como único Ingress, servicios en red privada	Un solo lugar donde verificar la firma y redirigir; superficie de ataque mínima	Una llamada más por request; el BFF conoce a todos los servicios
Validar el token contra IAM en cada servicio	El secreto reside solo en IAM y BFF; los roles se releen en cada request	Latencia y dependencia con IAM en cada request
contenido de archivos directo a S3 con URL pre-firmada	Ningún servicio recibe archivos; sin multipart ni límites de pod	El frontend valida tipo y tamaño; file-storage confía en él
Score sin pesos, máximo de dos coberturas	Explicable para el candidato ("tenés 6 de 9") y calculado en un solo lugar	No distingue skills centrales de accesorias
Similitud por tags ponderadas por rareza (IDF)	Comparar por concepto y no por palabra; umbral expresado en "tags típicas"	Depende de la calidad de las tags del LLM
Replay como reconciliación	Backend liviano, sin job ni retención	Entrega at-most-once; se ejecuta el replay antes de una demo
Read model propio en analytics	Reducir el acoplamiento	Consistencia eventual
Métricas al momento, snapshot solo para series	Sin watermarks que se desvíen sin que nadie lo note	Un hueco en la serie si el job no se ejecutó
Agente con juez y escritura en código	Nada entra al CV sin evidencia citada	De 3 a 7 llamadas al modelo por turno de edición

Pendientes identificados.[3]

Pendiente	Estado
Login con Google (OAuth 2.0 / OIDC)	A DEFINIR
Logout con revocación del refresh token en Redis	A DEFINIR
Validación de tipo y tamaño de archivo del lado de file-storage	A DEFINIR
lynq-email: servicio de notificaciones por correo	A DEFINIR
IAM con OTP como segundo factor	A DEFINIR
Lynq CD: despliegue continuo	EN ROADMAP
Juez con un modelo distinto al del editor	EN ROADMAP
Intención `unknown` en el agente de intención de CV Tailor: cuando no puede deducir si el mensaje es una edición o una consulta, pedirle al usuario que sea más claro en lugar de asumir `edit`	EN ROADMAP
Agente de IA de búsqueda autónoma	FUERA DE ALCANCE
Outreach inteligente al talent pool	FUERA DE ALCANCE
Métricas de calidad de contratación	FUERA DE ALCANCE

[3]Pendientes identificados al día de la fecha de este documento. Los ítems listados no son vinculantes ni implican obligación de implementarlos; su alcance y prioridad se definirán con los directores de carrera.